Guía Paso a Paso para la Resolución del Trabajo Final Integrador FESAT 2026

EJERCICIO 1: Sanitización y Análisis Estadístico de Sensores

Este bloque evalúa la capacidad de normalizar flujos de datos heterogéneos y extraer métricas consolidadas a partir de un conjunto de lecturas térmicas de la empresa *EcoTech Solutions*.

Paso 1.1: Diseñar la Función de Normalización (`limpiar_registro_sensor`)

El objetivo es crear un componente que reciba un único diccionario con datos crudos ("sucios") y retorne un nuevo diccionario perfectamente estandarizado bajo reglas estrictas de negocio.

1. **Definición e Inicialización:**
    - Declarar la función `limpiar_registro_sensor` especificando que recibe un diccionario como parámetro de entrada.
    - Inicializar una estructura de diccionario vacía dentro del cuerpo de la función. Aquí se construirán y almacenarán las claves limpias para evitar mutar los datos originales.
2. **Sanitización del Identificador (`sensor_id`):**
    - Extraer el valor asociado a la clave `"sensor_id"`.
    - Aplicar métodos de cadenas de texto (como `.strip()`) para **eliminar de forma drástica cualquier espacio en blanco sobrante** al inicio o al final del texto.
    - Almacenar el resultado limpio en el diccionario de retorno.
3. **Estandarización de la Temperatura (`temperatura`):**
    - Obtener el valor de la clave `"temperatura"`.
    - **Bifurcación por tipo de dato:** Evaluar si el valor ya es de tipo numérico (`int` o `float`). Si es así, realizar un casteo directo a `float`.
    - **Limpieza de cadenas:** Si el valor se presenta como un texto (ej. `"24.5C"`, `"31.2°C"`), se deben purgar los caracteres no numéricos. Utilizar de forma encadenada métodos de reemplazo (`.replace()`) para remover la letra `"C"`, la `"F"`, el símbolo de grado `"°"` y espacios remanentes.
    - **Control de fallos distributivos:** Envolver la conversión final a flotante dentro de una estructura de control de excepciones `try-except`. Si la cadena está corrupta y genera un error de valor (`ValueError`), la

captura del error debe asignar por defecto el valor numérico seguro `0.0`.

4. **Tratamiento del Atributo de Ubicación (`ubicacion`):**
    - Recuperar el valor de la clave `"ubicacion"`.
    - Validar mediante condicionales que el campo contenga texto real y no esté vacío ni constituido únicamente por espacios.
    - Si es válido, guardar la cadena limpia de espacios sobrantes. En caso de no haberse registrado o ser una cadena vacía, asignar el valor por defecto seguro `"Área No Asignada"`.
5. **Retorno de Datos:**
    - Finalizar la función devolviendo el diccionario estandarizado completamente construido.

Paso 1.2: Diseñar la Función de Mapeo Estadístico (`generar_resumen_sensores`)

Esta función procesará colecciones de datos previamente sanitizados para aislar anomalías térmicas y calcular indicadores globales.

1. **Estructura de Parámetros:**
    - Definir la función `generar_resumen_sensores` requiriendo dos argumentos: la lista de sensores limpios y un valor numérico flotante que actúe como umbral de temperatura de alerta.
2. **Cláusula de Guarda (Validación de Seguridad):**
    - Evaluar inmediatamente al inicio si la lista de sensores está vacía. Si no contiene elementos, retornar de forma anticipada una lista vacía y `0.0` como promedio, previniendo así errores críticos de división por cero (`ZeroDivisionError`) en la ejecución.
3. **Ciclo de Procesamiento y Acumulación:**
    - Inicializar un contador/acumulador flotante en `0.0` para la sumatoria térmica y una lista vacía para almacenar las alertas detectadas.
    - Implementar un bucle iterativo (`for`) para recorrer cada sensor de la lista.
    - En cada ciclo, extraer el valor numérico de la temperatura ya normalizada y sumarlo al acumulador global.
    - **Evaluación del Umbral:** Utilizar una estructura condicional (`if`) para comprobar si la temperatura del sensor actual supera de manera estricta el umbral de alerta. Si se cumple la condición, extraer el identificador del sensor (`sensor_id`) y añadirlo a la lista de alertas.
4. **Cálculo del Promedio Métrico:**
    - Dividir la sumatoria total de las temperaturas acumuladas entre la longitud total de la lista de sensores procesados.

    - Aplicar la función matemática integrada `round()` sobre el resultado para limitar la precisión decimal exactamente a **dos dígitos flotantes**.
5. **Retorno Estructurado:**
    - Finalizar la función devolviendo una **tupla** inmutable que contenga exactamente dos posiciones: la lista de identificadores en alerta y el valor del promedio redondeado.

EJERCICIO 2: Modelo de Productos, Excepciones y Persistencia

Este bloque evalúa el dominio de la Programación Orientada a Objetos (POO) avanzada, garantizando el encapsulamiento de datos críticos y el comportamiento polimórfico del catálogo de productos.

Paso 2.1: Diseñar la Excepción Personalizada (`StockInvalidoError`)

El objetivo es dar identidad propia a los errores de regla de negocio, impidiendo que el sistema registre inconsistencias operativas como stocks negativos.

1. **Estructura de la Clase de Error:**
    - Declarar la clase `StockInvalidoError` asegurando que herede directamente de la clase base nativa de Python `Exception`.
2. **Definición del Constructor Especializado:**
    - Diseñar el método inicializador `__init__`. Este debe aceptar un parámetro opcional para el mensaje de error, preconfigurando un texto descriptivo por defecto de forma clara (por ejemplo: `"El stock no puede ser un valor negativo."`).
    - Invocar al constructor de la superclase utilizando la instrucción `super().__init__(mensaje)` para transferir el texto y asegurar que las herramientas de diagnóstico nativas de Python procesen el error correctamente.

Paso 2.2: Diseñar la Clase Base del Catálogo (`DispositivoEcologico`)

Esta entidad centraliza el estado común de los productos sustentables y restringe el acceso directo a variables sensibles.

1. **Inicialización y Atributos Públicos:**
    - Crear la clase `DispositivoEcologico`.
    - En su método constructor `__init__`, requerir los datos esenciales de la entidad: `id_producto`, `nombre`, `precio` y `stock`. Asignar los tres primeros directamente a variables de instancia tradicionales.

2. **Implementación del Encapsulamiento del Stock:**
    - Crear de manera interna una variable de respaldo privada para el almacenamiento del stock utilizando la convención visual de privacidad de Python (un guion bajo al inicio, ej. `self._stock`), inicializada en `0`.
    - Definir un método de lectura decorado explícitamente con `@property` llamado `stock` que se limite a retornar el valor oculto de `self._stock`.
    - Diseñar su correspondiente método de escritura decorándolo con `@stock.setter`.
3. **Lógica de Validación en el Setter:**
    - Dentro del bloque del *setter*, evaluar mediante un condicional si el valor numérico entrante es inferior a `0`.
    - Si la condición es verdadera, interrumpir inmediatamente el flujo lanzando la excepción personalizada (`raise StockInvalidoError`) programada en el paso anterior, parametrizando un mensaje dinámico que exponga el nombre del producto y el valor erróneo intentado.
    - Si el valor es válido (cero o positivo), proceder a guardarlo formalmente en la variable protegida `self._stock`.
    - *Regla de Consistencia:* En el constructor principal de la clase, en lugar de asignar el parámetro directamente a la variable oculta, se debe invocar a la propiedad pública `self.stock = stock` para forzar a que la validación del setter se ejecute desde el primer instante de la creación del objeto.
4. **Diseño de la Ficha Técnica Base:**
    - Desarrollar el método `generar_ficha_tecnica`. Este no requiere parámetros adicionales y debe estructurar y retornar un diccionario de Python.
    - Las claves del diccionario deben representar los campos comerciales, formateando el atributo de precio como una cadena monetaria limpia con dos decimales de precisión (utilizando expresiones como `f"${self.precio:.2f}"`).

Paso 2.3: Diseñar la Subclase Especializada (`PanelSolar`)

Este componente demuestra los principios de reutilización mediante la extensión de clases (herencia) y la modificación de comportamientos estándar (polimorfismo).

1. **Declaración de Relación de Herencia:**
    - Crear la clase `PanelSolar` especificando explícitamente entre paréntesis que su clase madre es `DispositivoEcologico`.
2. **Extensión del Constructor de Subclase:**
    - Configurar el método `__init__` para capturar tanto los parámetros universales del catálogo como las propiedades de ingeniería específicas del panel fotovoltaico: `eficiencia` y `potencia_w`.

    - Invocar en la primera línea de ejecución el constructor de la clase superior a través de `super().__init__(...)` pasando los argumentos obligatorios correspondientes. Esto asegura la correcta inicialización del identificador, el nombre, el precio y las reglas nativas del stock.
    - Guardar localmente en variables de instancia los atributos nuevos de eficiencia y potencia.
3. **Sobrescritura Polimórfica del Reporte:**
    - Declarar el método `generar_ficha_tecnica` dentro de la estructura de la clase `PanelSolar` con la misma firma que el original.
    - Capturar el diccionario base generado por la clase madre invocando a `super().generar_ficha_tecnica()` y almacenándolo en una variable local.
    - Utilizar operaciones nativas de diccionarios (como el método `.update()`) para incorporar de forma limpia las nuevas claves del producto especializado (tales como el tipo específico de panel, la potencia nominal en Watts y su porcentaje de eficiencia energética).
    - Retornar la estructura de datos enriquecida final.

PASO 3: Validación General del Programa (Demostración Cooperativa)

- **Para el bloque estadístico:** Declarar una lista con diccionarios de prueba que emulen los casos de error reales planteados (cadenas con textos anexados tipo `"35.0 C"`, espacios flotantes y ubicaciones vacías). Invocar secuencialmente las funciones mediante comprensión de listas y validar con salidas impresas por consola (`print`) que el promedio térmico y la lista de alertas coinciden con las especificaciones.
- **Para el bloque de objetos:** Instanciar un objeto de la clase `PanelSolar` con parámetros correctos y verificar visualmente que su ficha técnica imprima el diccionario expandido polimórficamente. Posteriormente, estructurar un bloque `try-except` donde de forma intencional se intente asignar un stock de `-5` al objeto; el script debe demostrar que captura exitosamente la excepción `StockInvalidoError` e imprime un mensaje de advertencia controlado en la terminal en lugar de colapsar la aplicación.